

Format pliku z zadaniami (JSON)

Wersja na: 04.09.2026

Pomyśleliśmy, że dobrze byłoby móc kontrolować i zmieniać **kolejność zadań** w podsekcji. Żeby to połączyć z **wariantami zadań pod różne motywy** (zainteresowania), wymyśliliśmy takie rozwiązanie. To **propozycja** — może się jeszcze zmienić. Poniżej opisujemy, jak plik jest zbudowany i jak go wypełniać.

Wszystkie zadania siedzą w **jednym pliku** `.json`. Plik wypełnia się ręcznie, a potem wgrywa przez panel admina (zakładka „Import treści”). Czyta się go od góry do dołu jak spis treści kursu: **sekcje → podsekcje → zadania**.

Uwaga: opisy (`section_description` , `subsection_description`) zostały na razie usunięte. Skoro nazwa sekcji i podsekcji jest teraz podana tylko raz, opis też byłby jednorazowy — w razie potrzeby zawsze można go z powrotem dodać.

Uwaga: ta wersja formatu **nie obejmuje jeszcze**:

- modułu czasowego** — planowany jest jako kolejny `type` w tej samej strukturze (element listy `tasks`); szczegóły formatu do ustalenia,
- teorii / e-booków** — sposób zapisu jest jeszcze do ustalenia; krótkie bloki teorii między zadaniami mogłyby trafić do tego pliku, dłuższe materiały do czytania prawdopodobnie w osobnym pliku.

Na razie tych rzeczy się w pliku nie pisze.

Struktura

Cały plik to **lista sekcji**. Każda sekcja ma **listę podsekcji**, każda podsekcja ma **listę zadań**, a każde zadanie ma **warianty per motyw** (`themes`) — i dopiero w wariantie jest właściwa treść.

Nazwa sekcji i podsekcji jest wpisana **raz**, w nagłówku — nie przy każdym zadaniu.

Pola

Pole	Co to
section	nazwa sekcji (np. „Ułamki proste”)
subsections	lista podsekcji, w kolejności
subsection	nazwa podsekcji
tasks	lista zadań, w kolejności przechodzenia
type	rodzaj zadania: <code>single_choice</code> , <code>short_answer</code> albo <code>memory</code>
difficulty	trudność: 1, 2 albo 3 (przy <code>memory</code> się pomija)
themes	warianty treści per motyw; default musi być zawsze; wariant motywu podaje zwykle tylko prompt, resztę dziedziczy z default
prompt	treść pytania lub polecenia
options	zawsze dokładnie 3 opcje odpowiedzi w <code>single_choice</code>
text / correct	treść opcji; <code>correct</code> : <code>true</code> przy dokładnie jednej
answers	lista akceptowanych odpowiedzi w <code>short_answer</code>
pairs	pary w <code>memory</code> — 3 albo 6, każda z a i b

```

[
  {
    "section": "Nazwa pierwszej sekcji",
    "subsections": [
      {
        "subsection": "Nazwa pierwszej podsekcji",
        "tasks": [

          {
            "type": "single_choice",
            "difficulty": 1,
            "themes": {
              "default": {
                "prompt": "Treść pytania?",
                "options": [
                  { "text": "odpowiedź błędna" },
                  { "text": "odpowiedź poprawna", "correct": true },
                  { "text": "odpowiedź błędna" }
                ]
              },
              "sport": {
                "prompt": "To samo pytanie w wersji sportowej?"
              },
              "gry": {
                "prompt": "To samo pytanie w wersji dla gier?"
              },
              "zwierzeta": {
                "prompt": "To samo pytanie w wersji o zwierzętach?"
              }
            }
          },

          {
            "type": "short_answer",
            "difficulty": 2,
            "themes": {
              "default": {
                "prompt": "Treść pytania? Wpisz wynik.",
                "answers": ["poprawny zapis", "inny akceptowany zapis"]
              },
              "sport": {
                "prompt": "To samo pytanie w wersji sportowej? Wpisz wynik."
              },
              "gry": {
                "prompt": "To samo pytanie w wersji dla gier? Wpisz wynik."
              }
            }
          },

          {
            "type": "memory",
            "themes": {
              "default": {
                "prompt": "Połącz w pary.",
                "pairs": [
                  { "a": "lewa strona", "b": "prawa strona" },
                  { "a": "lewa strona", "b": "prawa strona" },
                  { "a": "lewa strona", "b": "prawa strona" }
                ]
              }
            }
          }
        ]
      }
    ]
  },
  {

```

```
        "subsection": "Nazwa drugiej podsekcji",
        "tasks": []
    }
  ],
  {
    "section": "Nazwa drugiej sekcji",
    "subsections": []
  }
]
```

Kolejność

Zależało nam na elastyczności — żeby dało się ułożyć kurs w dowolnej kolejności i łatwo ją zmieniać.

Kolejność = pozycja na liście. Wszędzie: sekcje, podsekcje, zadania.

- Żeby przesunąć zadanie, wystarczy przenieść jego blok `{ ... }` w inne miejsce listy `tasks`.
- Żeby dodać zadanie w środku, wkleja się blok tam, gdzie ma być.
- Nie ma żadnych numerów do pilnowania.

Kolejność **nie zależy** od trudności ani typu — zadania układa się w takiej kolejności, w jakiej mają się pojawiać. Zwykle łatwiejsze najpierw, ale nie jest to narzucone; można też np. dać najpierw trzy zamknięte, potem otwarte, potem memory.

Motywy (themes)

Motyw to zainteresowanie ucznia (wybiera je raz na początku). Każde zadanie ma obiekt `themes`: kluczem jest nazwa motywu, wartością — treść zadania dla danego motywu.

Dostępne motywy:

default sport gry lego zwierzeta rysowanie muzyka jedzenie

W pliku używa się **tej krótkiej nazwy** (`muzyka`, `jedzenie`, `lego` ...). To techniczny identyfikator — w aplikacji uczeń widzi pełną etykietę zainteresowania (np. „Rysowanie i sztuka”, „Muzyka i taniec”, „Jedzenie i gotowanie”). W `themes` liczy się tylko ten krótki klucz i musi być zapisany dokładnie tak jak wyżej.

Zasady:

- **default jest obowiązkowy w każdym zadaniu.** To wersja, którą widzi każdy, kto nie ma dopasowanego motywu.
- Uczeń z motywem np. `sport` dostaje wariant `sport` **tylko na tych zadaniach, które mają klucz `sport`**. Na pozostałych widzi `default`.
- Zadanie „bez motywu” = zadanie, które ma **tylko default**. Widzą je wszyscy. To jest normalny przypadek — motyw dokładnie się tam, gdzie jest dla niego treść, reszta leci na `default`.

Co wpisać w wariancie motywu

Motyw zmienia **tylko fabułę**. Liczby, odpowiedzi i to, która odpowiedź jest poprawna, są takie same jak w `default`. Dlatego w wariancie motywu wpisuje się **tylko prompt** — `options` / `answers` / `pairs` dziedziczy się z `default`.

Import składa wariant tak: bierze `default` i nadpisuje go tym, co poda motyw. Czego motyw nie poda — zostaje z `default`.

Wyjątek: jeśli w danym motywie sama treść opcji jest inna (np. z jednostkami — „96 cm” zamiast „96”), motyw może podać własne `options`. Wtedy **zastępuje** całą listę dla tego motywu (nadal dokładnie 3 opcje, jedna `correct`). Albo dziedziczyć całe pole, albo podajesz całe — bez scalania po jednej opcji.

Zadanie z kilkoma motywami:

```
{
  "type": "single_choice",
  "difficulty": 2,
  "themes": {
    "default": {
      "prompt": "Antek wyznaczył sobie plan na popołudnie. Poszło mu lepiej, niż zakładał, i zrealizował 7/5 swojego planu. Jaki to rodzaj ułamka?",
      "options": [
        { "text": "właściwy" },
        { "text": "niewłaściwy", "correct": true },
        { "text": "równy 0" }
      ]
    },
    "gry": {
      "prompt": "Kuba planował przejść określoną liczbę poziomów w grze, ale tak dobrze mu szło, że wykonał 7/5 swojego planu. Jaki to rodzaj ułamka?"
    },
    "lego": {
      "prompt": "Maja zaplanowała zbudować część wielkiego zamku z klocków. Wciągnęła się jednak w budowanie i wykonała aż 7/5 zaplanowanej pracy. Jaki to rodzaj ułamka?"
    },
    "zwierzeta": {
      "prompt": "Podczas spaceru pies Fado miał przejść wyznaczoną trasę, ale miał tyle energii, że pokonał 7/5 zaplanowanego dystansu. Jaki to rodzaj ułamka?"
    },
    "rysowanie": {
      "prompt": "Ola zaplanowała, ile rysunków przygotuje do swojego miniportfolio. Miała mnóstwo pomysłów i wykonała 7/5 swojego planu. Jaki to rodzaj ułamka?"
    },
    "muzyka": {
      "prompt": "Bartek ćwiczył układ taneczny i miał wykonać określoną liczbę powtórzeń. Muzyka tak go wciągnęła, że zrobił 7/5 zaplanowanej liczby powtórzeń. Jaki to rodzaj ułamka?"
    },
    "jedzenie": {
      "prompt": "Zosia planowała przygotować określoną liczbę mini pizz na szkolne spotkanie. Gotowanie szło jej świetnie, więc przygotowała 7/5 zaplanowanej liczby porcji. Jaki to rodzaj ułamka?"
    }
  }
}
```

Tylko `default` ma `options`. Każdy motyw podaje samą fabułę, a opcje `właściwy` / `niewłaściwy` / `równy 0` (w tym to, że poprawna jest „niewłaściwy”) dziedziczą z `default`.

Nazwy zadań

Pole `title` **nie ma** — tytułów się nie wpisuje. Numer „Zadanie 1, 2, 3...” nadaje się automatycznie z pozycji zadania na liście `tasks`.

„Zadanie 1 w Sporcie i Zadanie 1 w Gotowaniu — czy to problem?”

Nie. To **nie są dwa osobne zadania** — to jedno zadanie (jeden wpis na liście `tasks`) z dwoma wariantami w `themes`. Numer nadaje się z pozycji na liście, a uczeń widzi tylko jeden wariant. Nie ma jak zobaczyć „Zadania 1 sportowego” i „Zadania 1 kuchennego” naraz.

Typy zadań

Pole `type` mówi, jakie to zadanie. Trzy typy:

`single_choice` — wybór jednej odpowiedzi

```
{
  "type": "single_choice",
  "difficulty": 1,
  "themes": {
    "default": {
      "prompt": "Ile to 2 × 4?",
      "options": [
        { "text": "6" },
        { "text": "8", "correct": true },
        { "text": "10" }
      ]
    }
  }
}
```

- `options` — **zawsze dokładnie 3 opcje** (aplikacja pokazuje je jako A, B, C).
- **Dokładnie jedna** opcja ma `"correct": true`. Przy błędnych `correct` się pomija — nie piszesz `"correct": false`.

`short_answer` — uczeń wpisuje odpowiedź

```
{
  "type": "short_answer",
  "difficulty": 2,
  "themes": {
    "default": {
      "prompt": "Ile to 1,2 + 0,3? Wpisz wynik.",
      "answers": ["1,5", "1.5"]
    }
  }
}
```

- `answers` — lista akceptowanych zapisów odpowiedzi. Jeśli odpowiedź to ułamek dziesiętny, podaje się wersję **z przecinkiem i z kropką** (`"1,5"` oraz `"1.5"`), bo dziecko może wpisać jedno albo drugie.
- System przymyka oko na spacje i wielkość liter.

`memory` — łączenie w pary

```
{
  "type": "memory",
  "themes": {
    "default": {
      "prompt": "Połącz działania z wynikami.",
      "pairs": [
        { "a": "2 × 3", "b": "6" },
        { "a": "4 × 2", "b": "8" },
        { "a": "5 × 2", "b": "10" }
      ]
    }
  }
}
```

- `pairs` — **3 pary** (wersja łatwa, 6 kafelków) albo **6 par** (wersja trudniejsza, 12 kafelków). Nic pomiędzy.
- Każda para to dwie rzeczy, które do siebie pasują (`a` i `b`). Z każdej pary powstają 2 kafelki, przetasowane.
- `memory` **nie ma pola `difficulty`** — o trudności decyduje liczba par.
- Motywy: zwykle `memory` ma sam `default`, ale w razie potrzeby może mieć wariant motywu — podaje wtedy tylko `prompt`, `pairs` dziedziczy z `default`.

Trudność (`difficulty`)

Liczba 1, 2 albo 3 przy każdym zadaniu (poza memory). To etykieta „jak trudne jest to konkretne zadanie” — używana m.in. przez test poziomujący. **Nie wpływa na kolejność** zadań w podsekcji; o kolejności decyduje pozycja na liście.

Zasady treści

- **Bez nazw marek.** Zamiast „Minecraft”, „LEGO” itp. — ogólnie: „w grze”, „kryształy”, „klocki”.
 - Ułamki dziesiętne z przecinkiem (0,5), jak w polskiej szkole.
-

Jak budować plik — krok po kroku

1. Plik zaczyna się od tablicy [].
 2. W środku — obiekt sekcji: "section" + "subsections": [] .
 3. W subsections — obiekt podsekcji: "subsection" + "tasks": [] .
 4. W tasks — zadania **w kolejności**, w jakiej uczeń ma je przechodzić.
 5. Każde zadanie ma: type, difficulty (poza memory), themes z **obowiązkowym default** .
 6. Wariant motywu to kolejny klucz w themes — podaje zwykle tylko prompt; options / answers / pairs dziedziczy z default .
 7. Kolejne podsekcje, kolejne sekcje — wszystko w kolejności listy.
-

Edycja i wgrywanie

- **Plik to źródło prawdy.** Poprawka = edycja pliku i ponowne wgranie przez „Import treści”.
- Docelowo wgranie pliku **podmienia** treść działów, których dotyczy — bez dokładania duplikatów (obecny import tylko dokłada; podmianę trzeba jeszcze zbudować razem z tym formatem).
- Usunięcie zadania z pliku = zadanie znika, a jego dotychczasowe odpowiedzi przestają się liczyć.

Identyfikatory zadań a postępy uczniów

Żeby ponowny import nie gubił postępów uczniów przy **poprawianiu treści** ani przy **zmianie kolejności**, każde zadanie potrzebuje stałego id. Nie wpisuje się go ręcznie:

1. Plik pisze się **bez id** (dokładnie jak w przykładach powyżej).
2. Pierwszy import nadaje każdemu zadaniu id i zwraca plik z dopisanymi id .
3. Od tej pory edytuje się **ten plik (już z id)** i wgrywa ponownie. Import dopasowuje zadania **po id, nie po pozycji** — więc bloki można dowolnie przestawiać bez utraty postępów.

W praktyce jeden dodatkowy krok: po pierwszym imporcie pobiera się plik z id i dalej pracuje się na nim.

Jeden plik, nie trzy

Nie potrzeba osobnych plików „zamknięte / otwarte / memory”. Wszystkie typy siedzą w jednej strukturze i rozróżnia je pole type. Dzięki temu w podsekcji łatwo ustawić dowolną kolejność między typami (np. zamknięte → memory → otwarte) — przy trzech osobnych plikach byłoby to trudniejsze.

Na przyszłość — plik teraz, panel później

Nie da się mieć naraz „plik nadpisuje wszystko” i „ręczne zmiany w panelu zostają” — to sprzeczność. Musi być jedno źródło prawdy. Dlatego proponujemy podzielić to w czasie:

Teraz (budowanie treści, brak realnych uczniów): plik JSON to główny sposób pracy nad treścią — układanie kolejności, warianty motywów, dodawanie zadań w konkretnym miejscu. Panel pozwala już edytować, dodawać i usuwać pojedyncze zadania, ale strukturę i kolejność wygodniej trzymać w pliku. Poprawka = edycja pliku i ponowne wgranie.

Później (będą prawdziwi uczniowie): edycja może się w całości przenieść do panelu, a ponowny import stanie się rzadki / awaryjny. Plik JSON zostaje wtedy jako „zrzut początkowy” / historia. Wgranie pliku po tym momencie to świadoma akcja „zresetuj do tego pliku” — za potwierdzeniem.